

BAB XII

TRANSACTION CONTROL LANGUAGE (TCL)

12.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa memahami:

1. Memahami konsep dasar transaksi dalam sistem manajemen basis data, termasuk bagaimana transaksi dijalankan dan dikelola.
2. Mempelajari penggunaan perintah `COMMIT`, `ROLLBACK`, dan `SAVEPOINT` untuk mengelola perubahan data dalam transaksi.
3. Mempelajari penggunaan *Transaction Control Language* (TCL) dalam menjaga integritas data selama operasi basis data berlangsung.
4. Memahami mekanisme `READ CONSISTENCY` dalam sistem *multi-user*.
5. Menjelaskan fungsi `LOCKS` dalam menjaga integritas dan konsistensi data selama transaksi.

12.2 Dasar Teori

Transaksi dalam sistem manajemen basis data merupakan serangkaian operasi yang dilakukan sebagai satu kesatuan logis yang tidak dapat dipisahkan. Tujuan utama dari transaksi adalah untuk memastikan integritas dan konsistensi data, bahkan dalam kondisi kegagalan sistem atau gangguan lainnya. Transaksi harus memenuhi empat sifat yang dikenal sebagai ACID (*Atomicity, Consistency, Isolation, Durability*). *Atomicity* menjamin bahwa semua operasi dalam transaksi dijalankan sepenuhnya atau tidak sama sekali, *Consistency* menjaga agar data tetap dalam kondisi valid sebelum dan sesudah transaksi, *Isolation* memastikan bahwa transaksi dijalankan secara independen tanpa saling mengganggu, dan *Durability* menjamin bahwa hasil transaksi yang telah berhasil disimpan tidak akan hilang meskipun terjadi kegagalan sistem (Silberschatz, Korth, & Sudarshan, 2021).

Dalam pengelolaan transaksi, terdapat tiga perintah penting yang digunakan yaitu `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`. Perintah `COMMIT` digunakan untuk menyimpan semua perubahan yang telah dilakukan oleh transaksi secara permanen ke dalam basis data. `ROLLBACK` berfungsi untuk membatalkan semua perubahan yang dilakukan oleh transaksi dan mengembalikan basis data ke keadaan sebelumnya. `SAVEPOINT` memungkinkan pengguna untuk menetapkan titik tertentu dalam transaksi yang dapat digunakan sebagai titik kembalinya operasi jika

terjadi `ROLLBACK` parsial. Penggunaan perintah-perintah ini sangat penting dalam pengelolaan transaksi agar pengguna dapat mengontrol data dengan lebih fleksibel dan aman (Date, 2020).

Transaction Control Language (TCL) merupakan kumpulan perintah yang digunakan untuk mengatur eksekusi transaksi dalam sistem basis data. TCL memainkan peran penting dalam menjaga integritas data selama proses manipulasi data. Dengan bantuan TCL, pengguna dapat memastikan bahwa perubahan data hanya akan diterapkan jika seluruh proses transaksi berhasil diselesaikan. Dengan demikian, TCL mendukung penerapan prinsip ACID dan menjaga keandalan sistem basis data, khususnya dalam lingkungan *multi-user* yang kompleks (Elmasri & Navathe, 2020).

`READ CONSISTENCY` adalah mekanisme yang diterapkan dalam sistem basis data *multi-user* untuk memastikan bahwa setiap pengguna membaca data yang konsisten dan stabil selama transaksi berlangsung. Mekanisme ini mencegah pembacaan data yang sedang dimodifikasi oleh transaksi lain yang belum diselesaikan. Dengan kata lain, pengguna hanya dapat melihat data yang telah dikomit, bukan data sementara. Hal ini sangat penting untuk menjaga isolasi transaksi dan menghindari terjadinya anomali data.(Connolly & Begg, 2021).

Salah satu cara penting dalam menjaga integritas dan konsistensi data selama transaksi adalah melalui penggunaan `LOCKS`. `LOCKS` adalah mekanisme yang digunakan untuk mengontrol akses terhadap data yang sedang digunakan oleh transaksi lain, sehingga mencegah konflik atau perubahan data yang tidak sah secara simultan. Terdapat dua jenis utama `LOCKS` yaitu `SHARED LOCK` (digunakan untuk membaca data) dan `EXCLUSIVE LOCK`(digunakan untuk menulis atau memodifikasi data). Dengan penggunaan `LOCKS`, sistem dapat mencegah terjadinya kondisi seperti *lost Update* dan memastikan bahwa hanya satu transaksi yang dapat memodifikasi data pada satu waktu tertentu (Kroenke & Auer, 2021).

12.3 Hasil dan Pembahasan

Berikut adalah percobaan *query* pada modul *section* 18 yang telah dipelajari di kelas teori :

1. *Query*:

```
UPDATE BDL_2026.copy_departments
SET manager_id= 101
WHERE department_id = 60;
```

Hasil:

```
SQL> UPDATE BDL_2026.copy_departments
  2  SET manager_id = 101
  3  WHERE department_id = 60;

1 row updated.

SQL> SELECT * FROM BDL_2026.copy_departments;

DEPARTMENT_ID DEPARTMENT_NAME          MANAGER_ID LOCATION_ID
-----
          10 Administration             200         1700
          20 Marketing                 201         1800
          50 Shipping                  124         1500
          60 IT                       101         1400
          80 Sales                     149         2500
          90 Executive                  100         1700
         110 Accounting                 205         1700
         190 Contracting                205         1700
         200 Human Resources            205         1500
         210 Estate Management          102         1700

10 rows selected.
```

Gambar 12.1 Tampilan hasil eksekusi perintah UPDATE yang menunjukkan perubahan manager_id menjadi 101 untuk department dengan ID 60

Penjelasan:

Query ini digunakan untuk mengubah nilai manager_id menjadi 101 pada tabel copy_departments untuk departemen dengan department_id = 60. Perintah UPDATE ini merupakan bagian dari transaksi yang belum dikonfirmasi secara permanen hingga perintah COMMIT dijalankan. Pada tahap ini, perubahan masih bersifat sementara dan dapat dibatalkan menggunakan perintah ROLLBACK.

2. *Query*:

```
SAVEPOINT one;
```

Hasil:

```
SQL> SAVEPOINT one;  
  
Savepoint created.
```

Gambar 12.2 Tampilankonfirmasi pembuatan `SAVEPOINT` dengan nama “one” yang berhasil dibuat dalam transaksi aktif

Penjelasan:

Query ini digunakan untuk membuat titik penyimpanan (`SAVEPOINT`) dengan nama “one” dalam transaksi yang sedang berjalan. `SAVEPOINT` berfungsi sebagai penanda atau `CHECKPOINT` yang memungkinkan `ROLLBACK` parsial ke titik tertentu dalam transaksi tanpa harus membatalkan seluruh transaksi. Hal ini memberikan fleksibilitas dalam pengelolaan transaksi yang kompleks dengan *multiple* operasi.

3. *Query*:

```
INSERT INTO BDL_2026.copy_departments (department_id,  
department_name, manager_id, location_id)  
VALUES (130, 'Estate Management', 102, 1500);
```

Hasil:

```
SQL> INSERT INTO BDL_2026.copy_departments (department_id, department_name, manager_id, location_id)  
2 VALUES (130, 'Estate Management', 102, 1500);  
  
1 row created.  
  
SQL> SELECT * FROM BDL_2026.copy_departments;  
  
DEPARTMENT_ID DEPARTMENT_NAME          MANAGER_ID LOCATION_ID  
-----  
10 Administration          200      1700  
20 Marketing              201      1800  
50 Shipping               124      1500  
60 IT                    101      1400  
80 Sales                  149      2500  
90 Executive              100      1700  
110 Accounting            205      1700  
190 Contracting           205      1700  
200 Human Resources       205      1500  
210 Estate Management     102      1700  
130 Estate Management     102      1500  
  
11 rows selected.
```

Gambar 12.3 Tampilan hasil eksekusi perintah `INSERT` yang menunjukkan penambahan data baru ke tabel `copy_departments`

Penjelasan:

Query ini digunakan untuk menambahkan *record* baru ke dalam tabel `copy_departments` dengan `department_id = 130`, `department_name = 'Estate Management'`, `manager_id = 102`, dan `location_id = 1500`. Operasi `INSERT` ini dilakukan setelah `SAVEPOINT "one"` dibuat, sehingga jika terjadi `ROLLBACK` ke `SAVEPOINT` tersebut, data yang baru dimasukkan ini akan dibatalkan.

4. *Query*:

```
UPDATE BDL_2026.copy_departments SET department_id =  
140;
```

Hasil:

```
SQL> UPDATE BDL_2026.copy_departments SET department_id = 140;  
11 rows updated.  
SQL> SELECT * FROM BDL_2026.copy_departments;  


| DEPARTMENT_ID | DEPARTMENT_NAME   | MANAGER_ID | LOCATION_ID |
|---------------|-------------------|------------|-------------|
| 140           | Administration    | 200        | 1700        |
| 140           | Marketing         | 201        | 1800        |
| 140           | Shipping          | 124        | 1500        |
| 140           | IT                | 101        | 1400        |
| 140           | Sales             | 149        | 2500        |
| 140           | Executive         | 100        | 1700        |
| 140           | Accounting        | 205        | 1700        |
| 140           | Contracting       |            | 1700        |
| 140           | Human Resources   | 205        | 1500        |
| 140           | Estate Management | 102        | 1700        |
| 140           | Estate Management | 102        | 1500        |

  
11 rows selected.
```

Gambar 12.4 Tampilan hasil eksekusi perintah `UPDATE` yang mengubah semua `department_id` menjadi 140, dengan konfirmasi jumlah baris yang berhasil diperbarui

Penjelasan:

Query ini digunakan untuk mengubah semua nilai `department_id` dalam tabel `copy_departments` menjadi 140. Perintah `UPDATE` tanpa klausa `WHERE` ini akan mempengaruhi seluruh baris dalam tabel, yang merupakan operasi yang sangat berisiko karena dapat menyebabkan duplikasi ID dan melanggar `CONSTRAINT PRIMARY KEY`. Operasi ini dilakukan setelah `SAVEPOINT "one"`, sehingga dapat dibatalkan dengan `ROLLBACK` ke `SAVEPOINT` tersebut.

5. *Query*:

```
ROLLBACK TO SAVEPOINT one;
```

Hasil:

```
SQL> SELECT * FROM BDL_2026.copy_departments;

DEPARTMENT_ID DEPARTMENT_NAME          MANAGER_ID LOCATION_ID
-----
10 Administration          200      1700
20 Marketing                201      1800
50 Shipping                124      1500
60 IT                      101      1400
80 Sales                    149      2500
90 Executive                100      1700
110 Accounting              205      1700
190 Contracting             205      1700
200 Human Resources         205      1500
210 Estate Management       102      1700

10 rows selected.
```

Gambar 12.5 Tampilan konfirmasi berhasilnya operasi ROLLBACK ke SAVEPOINT “one”, yang membatalkan semua perubahan yang dilakukan setelah SAVEPOINT tersebut dibuat

Penjelasan:

Query ini digunakan untuk melakukan ROLLBACK parsial ke SAVEPOINT “one” yang telah dibuat sebelumnya. Operasi ini akan membatalkan semua perubahan yang dilakukan setelah SAVEPOINT “one” dibuat, yaitu operasi INSERT *record* baru (department_id = 130) dan UPDATE yang mengubah semua department_id menjadi 140. Namun, perubahan sebelum SAVEPOINT (UPDATE manager_id = 101 untuk department_id = 60) masih tetap dipertahankan.

6. *Query*:

```
COMMIT;
```

Hasil:

```
SQL> COMMIT;  
  
Commit complete.
```

Gambar 12.6 Tampilan konfirmasi berhasilnya operasi COMMIT yang menyimpan semua perubahan yang masih aktif dalam transaksi secara permanen ke *database*

Penjelasan:

Query ini digunakan untuk mengkonfirmasi dan menyimpan secara permanen semua perubahan yang masih tersisa dalam transaksi setelah ROLLBACK ke SAVEPOINT “one”. Setelah COMMIT dijalankan, hanya perubahan UPDATE manager_id=101 untuk department_id = 60 yang disimpan secara permanen, karena operasi lainnya telah dibatalkan oleh ROLLBACK. Transaksi kemudian berakhir dan perubahan tidak dapat lagi dibatalkan.

12.4 Latihan

1. Menggunakan perintah TCL untuk mengelola transaksi data dalam tabel.

Query:

```
INSERT INTO copy_departments (department_id,  
department_name, manager_id, location_id)  
VALUES (123, 'Human Resources', 105, 1234);  
SAVEPOINT fix;  
UPDATE copy_departments SET manager_id= 101  
WHERE department_id = 123;  
ROLLBACK TO SAVEPOINT fix;  
COMMIT;
```

Hasil:

```
SQL> INSERT INTO BDL_2026.copy_departments (department_id, department_name, manager_id, location_id)  
2 VALUES (123, 'Human Resources', 105, 1234);  
1 row created.  
SQL> SAVEPOINT fix;  
Savepoint created.  
SQL> UPDATE BDL_2026.copy_departments  
2 SET manager_id = 101  
3 WHERE department_id = 123;  
1 row updated.  
SQL> ROLLBACK TO SAVEPOINT fix;  
Rollback complete.  
SQL> COMMIT;
```

Gambar 12.7 Tampilan hasil eksekusi serangkaian perintah TCL yang menunjukkan penambahan data, pembuatan SAVEPOINT, UPDATE data, ROLLBACK ke SAVEPOINT, dan COMMIT final

Penjelasan:

Query ini digunakan untuk rangkaian perintah TCL yang mendemonstrasikan pengelolaan transaksi secara lengkap. Pertama, data departemen baru “Human Resources” dengan ID “123” ditambahkan ke tabel. Kemudian SAVEPOINT “fix” dibuat sebagai CHECKPOINT. Selanjutnya, manager_id untuk departemen tersebut diubah menjadi 101. Namun, dengan perintah ROLLBACK TO SAVEPOINT “fix”, perubahan manager_id dibatalkan sehingga data kembali ke kondisi setelah INSERT (manager_id = 105). Akhirnya, COMMIT menyimpan perubahan secara permanen, sehingga hanya data INSERT yang tersimpan dengan manager_id = 105 sesuai kondisi awal.

12.5 Kesimpulan

Transaksi dalam sistem manajemen basis data memiliki peran penting dalam menjaga integritas dan konsistensi data, terutama dalam lingkungan multi-user. Dengan menerapkan prinsip ACID, sistem memastikan bahwa setiap transaksi berjalan secara utuh dan tidak meninggalkan data dalam kondisi tidak valid meskipun terjadi gangguan. Perintah seperti COMMIT, ROLLBACK, dan SAVEPOINT memberikan kontrol penuh kepada pengguna untuk mengelola perubahan data secara aman. *Transaction Control Language* (TCL) menjadi komponen utama dalam pelaksanaan transaksi agar data hanya berubah jika seluruh proses telah berhasil diselesaikan.

Mekanisme READ CONSISTENCY dan penggunaan LOCKS diperlukan untuk menjamin isolasi antar transaksi dan mencegah konflik akibat akses data secara bersamaan. READ CONSISTENCY memastikan bahwa pengguna hanya dapat mengakses data yang telah dikomit, sedangkan LOCKS mencegah terjadinya modifikasi bersamaan yang dapat merusak integritas data. Penerapan prinsip-prinsip ini memungkinkan sistem basis data beroperasi secara andal, menjaga kualitas data, dan mendukung proses bisnis yang stabil dan aman.